AWS — Beginner to Practical

Complete Guide: All 12 Core Services with Real Examples

> **Java SDK v2 · AWS CLI v2 · Spring Boot · Free-tier eligible** Every service explained with architecture flow, bullet-point concepts, working Java code, and CLI commands.

Table of Contents

#	Service	Category	Free Tier
1	EC2	Virtual Servers	750 hrs/month (12 months)
2	S3	File Storage	5 GB forever
3	Lambda	Serverless Compute	1M requests forever
4	DynamoDB	NoSQL Database	25 GB forever
5	SQS	Message Queue	1M requests forever
6	SNS	Pub/Sub	1M publishes forever
7	API Gateway	HTTP Endpoints	1M calls (12 months)
8	CloudWatch	Observability	10 metrics + 5 GB logs forever
9	IAM	Security	Always free
10	CloudFormation	IaC	Always free
11	RDS	Managed SQL	750 hrs/month (12 months)
12	ELB + Auto Scaling	High Availability	750 hrs ELB (12 months)

---

1. EC2 — Elastic Compute Cloud

What is it?

- A **virtual server** (computer) running in AWS data centres
- You choose the **OS** (Amazon Linux, Ubuntu, Windows), **CPU, RAM, and storage**
- Your Spring Boot app runs on EC2 **exactly like on your laptop** — but always online
- You pay **per hour** the instance runs — stop it, stop billing

Architecture

```
User's browser
    │
    ▼  (HTTPS / HTTP)
Security Group  ←── firewall: only allow ports 22 (SSH) + 8080 (app)
    │
    ▼
EC2 Instance (t2.micro)
    │
    ▼
Spring Boot app running on :8080
```

Key concepts

Term	Meaning
AMI	Amazon Machine Image — pre-built OS snapshot. You pick one and EC2 boots from it
Instance type	t2.micro = 1 vCPU + 1 GB RAM. Free tier. Good for learning
Key pair	SSH certificate (.pem file). Download it — you need it to log in

Term	Meaning
Security group	Cloud firewall. Open port 22 (SSH) and your app port. Block everything else
Public IP	IP address assigned when instance starts — use this to access your app
EBS	Elastic Block Store — the hard disk attached to your EC2 instance

Step-by-step: Launch and deploy Spring Boot

Step 1 — Launch via AWS CLI

```
aws ec2 run-instances \
  --image-id ami-0c55b159cbfafe1f0 \
  --instance-type t2.micro \
  --key-name my-key-pair \
  --security-group-ids sg-0123456789abcdef0 \
  --tag-specifications
'ResourceType=instance,Tags=[{Key=Name,Value=MySpringApp}]'



# Check it is running
aws ec2 describe-instances \
  --filters "Name=tag:Name,Values=MySpringApp" \
  --query 'Reservations[].Instances[].{State:State.Name,IP:PublicIpAddress}'
```

Step 2 — Connect via SSH

```
ssh -i my-key-pair.pem ec2-user@<your-public-ip>
```

Step 3 — Install Java 17

```
sudo yum update -y
sudo yum install java-17-amazon-corretto -y
```

```
java -version
```

Step 4 — Upload your JAR (from your local machine)

```
scp -i my-key-pair.pem myapp.jar ec2-user@<your-ip>:/home/ec2-user/
```

Step 5 — Run your Spring Boot app

```
java -jar myapp.jar --server.port=8080 &

# Test it

curl http://<your-public-ip>:8080/api/health
```

> **Free tier:** t2.micro — 750 hours/month free for the first 12 months

---

2. S3 — Simple Storage Service

What is it?

- **Unlimited cloud file storage** organised in "buckets"
- Store any file: **images, PDFs, videos, CSVs, ZIP archives, JAR files, backups**
- Think of it as a **hard drive in the cloud that never fills up**
- Files get a unique URL — make them public or keep them private
- Built-in **99.999999999% durability** — your file is replicated across 3+ data centres

Architecture

```
User uploads profile photo

        │

        ▼

Spring Boot controller (receives multipart file)

        │

        ▼  (AWS SDK v2)
```

```
s3.putObject()

        │

        ▼

S3 Bucket: my-app-bucket

   ├── profile-images/user123.jpg   ← stores here

   ├── reports/march-2025.pdf

   └── backups/db-2025-04-30.sql



        │ URL returned

        ▼

Database stores:
"https://my-app-bucket.s3.amazonaws.com/profile-images/user123.jpg"
```

Maven dependency

```
<dependency>

    <groupId>software.amazon.awssdk</groupId>

    <artifactId>s3</artifactId>

    <version>2.21.0</version>

</dependency>
```

Java — complete CRUD service

```
@Service

public class S3Service {
```

```
    private final S3Client s3 = S3Client.builder()
            .region(Region.US_EAST_1)
            .build();

    private static final String BUCKET = "my-app-bucket";

    // ── UPLOAD ──────────────────────────────────────────────────────────
    public String upload(String key, byte[] data, String contentType) {
        s3.putObject(
            PutObjectRequest.builder()
                .bucket(BUCKET)
                .key(key)                          // e.g. "profile-images/user123.jpg"
                .contentType(contentType)          // e.g. "image/jpeg"
                .build(),
            RequestBody.fromBytes(data));

        // Return the public URL
        return String.format("https://%s.s3.amazonaws.com/%s", BUCKET, key);
    }

    // ── DOWNLOAD ────────────────────────────────────────────────────────
```

```
    public byte[] download(String key) {
        ResponseBytes<GetObjectResponse> object = s3.getObjectAsBytes(
            GetObjectRequest.builder()
                .bucket(BUCKET)
                .key(key)
                .build());
        return object.asByteArray();
    }


    // ── LIST ALL FILES ──────────────────────────────────────────────
    public List<String> listFiles(String prefix) {
        return s3.listObjectsV2(
            ListObjectsV2Request.builder()
                .bucket(BUCKET)
                .prefix(prefix)                    // e.g. "profile-images/"
                .build())
            .contents()
            .stream()
            .map(S3Object::key)
            .toList();
    }
```

```
    // ── DELETE ──────────────────────────────────────────────────────────
    public void delete(String key) {
        s3.deleteObject(
            DeleteObjectRequest.builder()
                .bucket(BUCKET)
                .key(key)
                .build());
    }

    // ── GENERATE PRESIGNED URL (time-limited download link) ─────────────
    public String presignedUrl(String key, int expiryMinutes) {
        S3Presigner presigner = S3Presigner.builder()
                .region(Region.US_EAST_1).build();

        GetObjectPresignRequest request = GetObjectPresignRequest.builder()
                .signatureDuration(Duration.ofMinutes(expiryMinutes))
                .getObjectRequest(r -> r.bucket(BUCKET).key(key))
                .build();

        return presigner.presignGetObject(request).url().toString();
    }
}
```

AWS CLI — common S3 commands

```
# Create a bucket
aws s3 mb s3://my-app-bucket

# Upload a file
aws s3 cp myfile.pdf s3://my-app-bucket/reports/

# List all files in a bucket
aws s3 ls s3://my-app-bucket --recursive

# Download a file
aws s3 cp s3://my-app-bucket/reports/myfile.pdf ./local-copy.pdf

# Delete a file
aws s3 rm s3://my-app-bucket/reports/myfile.pdf

# Sync a local folder to S3
aws s3 sync ./dist/ s3://my-app-bucket/frontend/
```

> **Free tier:** 5 GB storage + 20,000 GET requests/month — always free

---

3. Lambda — Serverless Functions

What is it?

- **Run code without managing a server** — upload a function, AWS runs it
- Only runs **when triggered** — zero cost when idle (unlike EC2 which bills per hour)
- Scales to **millions of invocations automatically**
- Max runtime: **15 minutes** per invocation — designed for short tasks
- Billed per **100ms of execution** — extremely cheap for bursty workloads

Five trigger types

```
HTTP request (API Gateway)     ──────┐

File uploaded to S3            ──────┤

                                     ├──▶  Lambda function  ──▶  your Java
code runs

Message in SQS queue           ──────┤

Scheduled timer (EventBridge)  ──────┤

DynamoDB stream change         ──────┘
```

Java — two handler patterns

Pattern 1: S3 trigger (resize image when file uploaded)

```
public class ImageResizeHandler

        implements RequestHandler<S3Event, String> {




    @Override

    public String handleRequest(S3Event event, Context context) {
```

```
        // Get details of the file that triggered this Lambda
        String bucket = event.getRecords().get(0)
                .getS3().getBucket().getName();
        String key    = event.getRecords().get(0)
                .getS3().getObject().getKey();

        context.getLogger().log("Processing: " + bucket + "/" + key);

        // Download, resize, re-upload
        byte[] original = downloadFromS3(bucket, key);
        byte[] resized  = resizeImage(original, 300, 300);
        uploadToS3(bucket, "thumbnails/" + key, resized);

        return "Processed: " + key;
    }
}
```

Pattern 2: HTTP trigger via API Gateway

```
public class UserHandler
        implements RequestHandler<APIGatewayProxyRequestEvent,
                                  APIGatewayProxyResponseEvent> {

    @Override
```

```
    public APIGatewayProxyResponseEvent handleRequest(
            APIGatewayProxyRequestEvent request, Context ctx) {

        String httpMethod = request.getHttpMethod();     // GET, POST, etc.
        String path       = request.getPath();           // /users/123
        String body       = request.getBody();           // JSON body (POST)

        Map<String, String> params =
            request.getQueryStringParameters();          // ?name=Suresh

        // Your logic here
        String responseBody = """
                { "message": "Hello, %s!" }
                """.formatted(params.getOrDefault("name", "World"));

        return new APIGatewayProxyResponseEvent()
                .withStatusCode(200)
                .withHeaders(Map.of("Content-Type", "application/json"))
                .withBody(responseBody);
    }
}
```

Deploy and test via CLI

```
# Package your function as a JAR
mvn package -DskipTests


# Create the Lambda function
aws lambda create-function \
  --function-name MyFunction \
  --runtime java17 \
  --handler com.example.UserHandler::handleRequest \
  --role arn:aws:iam::123456789012:role/lambda-role \
  --zip-file fileb://target/myapp.jar \
  --timeout 30 \
  --memory-size 512


# Update code after changes
aws lambda update-function-code \
  --function-name MyFunction \
  --zip-file fileb://target/myapp.jar


# Invoke and test directly
aws lambda invoke \
  --function-name MyFunction \
```

```
  --payload '{"queryStringParameters":{"name":"Suresh"}}' \

  response.json



cat response.json
```

> **Free tier:** 1 million requests + 400,000 GB-seconds/month — always free

---

4. DynamoDB — NoSQL Database

What is it?

- **Fully managed NoSQL database** — no servers, no patching, no manual backups
- **Millisecond latency** at any scale — from 1 request/day to millions/second
- Data stored in **tables** with a mandatory **partition key**
- No fixed schema — **each row (item) can have different attributes**
- Scales automatically — no manual configuration for traffic spikes

Table structure

```
Table: Users




┌──────────────────┬──────────────┬──────────────────┬──────────────┐

│  userId (PK)     │  name        │  email           │  any field   │

│  (Partition Key) │              │                  │  (optional)  │

├──────────────────┼──────────────┼──────────────────┼──────────────┤

│  user-001        │  Suresh      │  s@example.com   │  age: 35     │

│  user-002        │  Priya       │  p@example.com   │  city: Mumbai │
```

```
│  user-003        │  Ravi         │  r@example.com   │  (no extras)   │

└──────────────────┴───────────────┴──────────────────┴────────────────┘
```

Java — complete CRUD with AWS SDK v2

```
@Service

public class DynamoUserService {



    private final DynamoDbClient db = DynamoDbClient.builder()

            .region(Region.US_EAST_1)

            .build();



    private static final String TABLE = "Users";



    // ── CREATE ──────────────────────────────────────────────────────

    public void saveUser(String userId, String name, String email) {

        db.putItem(PutItemRequest.builder()

            .tableName(TABLE)

            .item(Map.of(

                "userId",    AttributeValue.fromS(userId),

                "name",      AttributeValue.fromS(name),

                "email",     AttributeValue.fromS(email),

                "createdAt", AttributeValue.fromS(Instant.now().toString()),
```

```
                "active",    AttributeValue.fromBool(true)

            ))

            .build());

    }



    // ── READ ──────────────────────────────────────────────────────────

    public Map<String, AttributeValue> getUser(String userId) {

        GetItemResponse response = db.getItem(GetItemRequest.builder()

            .tableName(TABLE)

            .key(Map.of("userId", AttributeValue.fromS(userId)))

            .build());



        return response.hasItem() ? response.item() : null;

    }



    // ── UPDATE (partial — only changes email) ─────────────────────────

    public void updateEmail(String userId, String newEmail) {

        db.updateItem(UpdateItemRequest.builder()

            .tableName(TABLE)

            .key(Map.of("userId", AttributeValue.fromS(userId)))

            .updateExpression("SET email = :e, updatedAt = :t")

            .expressionAttributeValues(Map.of(
```

```
                ":e", AttributeValue.fromS(newEmail),
                ":t", AttributeValue.fromS(Instant.now().toString()))
            .build());
    }

    // ── DELETE ──────────────────────────────────────────────────────
    public void deleteUser(String userId) {
        db.deleteItem(DeleteItemRequest.builder()
            .tableName(TABLE)
            .key(Map.of("userId", AttributeValue.fromS(userId)))
            .build());
    }

    // ── QUERY (all users in a city — requires GSI) ──────────────────
    public List<Map<String, AttributeValue>> getUsersByCity(String city) {
        QueryResponse response = db.query(QueryRequest.builder()
            .tableName(TABLE)
            .indexName("city-index")               // Global Secondary Index
            .keyConditionExpression("city = :c")
            .expressionAttributeValues(Map.of(
                ":c", AttributeValue.fromS(city)
```

```
            ))

            .build());

        return response.items();

    }

}
```

AWS CLI — DynamoDB commands

```
# Create table

aws dynamodb create-table \

  --table-name Users \

  --attribute-definitions AttributeName=userId,AttributeType=S \

  --key-schema AttributeName=userId,KeyType=HASH \

  --billing-mode PAY_PER_REQUEST



# Put an item

aws dynamodb put-item \

  --table-name Users \

  --item '{"userId":{"S":"u001"},"name":{"S":"Suresh"},"email":
{"S":"s@test.com"}}'



# Get an item

aws dynamodb get-item \

  --table-name Users \
```

```
  --key '{"userId":{"S":"u001"}}'



# Delete item

aws dynamodb delete-item \

  --table-name Users \

  --key '{"userId":{"S":"u001"}}'
```

> **DynamoDB vs RDS:** Use DynamoDB for key-based lookups at massive scale. Use RDS for joins, complex SQL, and ACID transactions.
>
> **Free tier:** 25 GB storage + 25 read/write capacity units — always free

---

5. SQS — Simple Queue Service

What is it?

- **Message queue** that decouples two services
- **Producer** puts a message in → **Consumer** picks it up later
- If the consumer crashes, **messages wait safely** — nothing lost
- Each message stays until the consumer **explicitly deletes it**
- Supports up to **256 KB per message**

Architecture — order processing

```
User places order

        │

        ▼

Order Service (Producer)

        │  sendMessage(orderId, email, amount)
```

```
        ▼

┌──────────────────────────────────────┐

│         SQS Queue: OrderQueue        │

│  [msg1] [msg2] [msg3] ...            │  ← messages wait safely

│                                      │  ← if consumer is DOWN, nothing lost

└──────────────────────────────────────┘

        │  receiveMessage() → process → deleteMessage()

        ▼

Email Service (Consumer)

        │

        ▼

Sends confirmation email to user
```

Java — Producer (send) and Consumer (receive)

```
@Service

public class SqsService {



    private final SqsClient sqs = SqsClient.builder()

            .region(Region.US_EAST_1)

            .build();



    private static final String QUEUE_URL =
```

```
        "https://sqs.us-east-1.amazonaws.com/123456789/OrderQueue";



    // ── PRODUCER ──────────────────────────────────────────────────────

    public String sendOrderEvent(String orderId, String email, double amount) {

        String messageBody = """

                {

                  "orderId": "%s",

                  "email":   "%s",

                  "amount":  %.2f,

                  "status":  "PLACED",

                  "timestamp": "%s"

                }

                """.formatted(orderId, email, amount, Instant.now());



        SendMessageResponse response = sqs.sendMessage(

            SendMessageRequest.builder()

                .queueUrl(QUEUE_URL)

                .messageBody(messageBody)

                .messageGroupId("orders")       // for FIFO queues

                .delaySeconds(0)                // deliver immediately

                .build());
```

```
        return response.messageId();
    }

    // ── CONSUMER ──────────────────────────────────────────────────────
    public void processMessages() {
        // Long-poll: wait up to 20s for messages (saves API call cost)
        ReceiveMessageResponse response = sqs.receiveMessage(
            ReceiveMessageRequest.builder()
                .queueUrl(QUEUE_URL)
                .maxNumberOfMessages(10)         // batch: up to 10 at once
                .waitTimeSeconds(20)             // long-polling
                .build());

        for (Message message : response.messages()) {
            try {
                // Process the message
                processOrder(message.body());

                // IMPORTANT: Only delete AFTER successful processing
                // If you crash before this, the message reappears
automatically
                sqs.deleteMessage(DeleteMessageRequest.builder()
```

```
                    .queueUrl(QUEUE_URL)
                    .receiptHandle(message.receiptHandle())
                    .build());


            } catch (Exception e) {
                // Don't delete — SQS will redeliver after visibility timeout
                log.error("Processing failed for message {}: {}",
                    message.messageId(), e.getMessage());
            }
        }
    }
}
```

Spring Boot — automatic SQS listener

```
// With spring-cloud-aws-messaging dependency
@SqsListener("OrderQueue")
public void onOrderMessage(String messageBody) {
    // Automatically polled, deserialized, and deleted on success
    OrderEvent order = objectMapper.readValue(messageBody, OrderEvent.class);
    emailService.sendConfirmation(order.getEmail(), order.getOrderId());
    log.info("Processed order: {}", order.getOrderId());
}
```

> **Critical rule:** Delete the message ONLY after successful processing. If your app crashes, the message reappears after the visibility timeout (default 30s) and is retried.
>
> **Free tier:** 1 million requests/month — always free

---

6. SNS — Simple Notification Service

What is it?

- **Publish once, notify many** (pub/sub pattern)
- You publish ONE message to a **topic** — SNS delivers it to ALL subscribers
- Subscribers can be: **SQS queues, Lambda functions, email, SMS, HTTPS endpoints**
- Delivery is **push-based** — SNS pushes to subscribers (unlike SQS which is pull)
- Used for **event broadcasting** — one event triggers multiple downstream actions

Architecture — fan-out pattern

```
Order placed event

        │

        ▼  publish()

┌──────────────────────────┐

│   SNS Topic: OrderEvents │

└──────────┬───────────────┘

           │ fans out simultaneously to ALL subscribers

    ┌──────┼──────┬──────────────┐

    ▼      ▼      ▼              ▼

  SQS    Lambda   Email       HTTP

  Queue  function address     endpoint
```

```
  (email (analytics) (admin@  (webhook

  service)           company) partner)



All 4 receive the SAME message at the SAME time
```

Java — publish to SNS

```
@Service

public class SnsService {



    private final SnsClient sns = SnsClient.builder()

            .region(Region.US_EAST_1)

            .build();



    private static final String TOPIC_ARN =

        "arn:aws:sns:us-east-1:123456789012:OrderEvents";



    // ── PUBLISH ──────────────────────────────────────────────────────

    public String publishOrderEvent(String orderId, String status) {

        String message = """

                {

                  "orderId":   "%s",

                  "status":    "%s",
```

```
                "timestamp": "%s"
                }""".formatted(orderId, status, Instant.now());


        PublishResponse response = sns.publish(
            PublishRequest.builder()
                .topicArn(TOPIC_ARN)
                .subject("Order Event: " + status)  // used for email subscribers
                .message(message)
                .build());


        return response.messageId();
    }


    // ── SUBSCRIBE AN EMAIL ADDRESS ──────────────────────────────
    public String subscribeEmail(String email) {
        SubscribeResponse response = sns.subscribe(
            SubscribeRequest.builder()
                .topicArn(TOPIC_ARN)
                .protocol("email")
                .endpoint(email)
                .build());
```

```
        return response.subscriptionArn();  // "pending confirmation" until
user clicks

    }



    // ── SUBSCRIBE AN SQS QUEUE ──────────────────────────────────────

    public String subscribeQueue(String sqsQueueArn) {

        SubscribeResponse response = sns.subscribe(

            SubscribeRequest.builder()

                .topicArn(TOPIC_ARN)

                .protocol("sqs")

                .endpoint(sqsQueueArn)

                .build());

        return response.subscriptionArn();

    }

}



// In your Order controller — one call notifies everything:

@PostMapping("/orders")

public ResponseEntity<Order> placeOrder(@RequestBody OrderRequest request) {

    Order order = orderService.create(request);



    // One publish → email service + analytics Lambda + admin SMS all fire
```

```
    snsService.publishOrderEvent(order.getId(), "PLACED");




    return ResponseEntity.status(201).body(order);

}
```

SNS vs SQS — choosing the right one

Need	Use
Notify multiple systems simultaneously	**SNS**
One consumer processes each message reliably	**SQS**
Both — broadcast + reliable per-consumer retry	**SNS → SQS fan-out**

> **Best practice:** SNS → SQS fan-out. SNS fans out to multiple SQS queues. Each queue has its own consumer. You get broadcast AND reliable retry per consumer.
>
> **Free tier:** 1 million publishes + 1,000 email notifications/month — always free

---

7. API Gateway

What is it?

- **Creates HTTP/HTTPS endpoints** that route requests to Lambda, EC2, or other services
- Handles **authentication, rate limiting, SSL/TLS, and CORS** — your backend does not have to
- Every public API needs a **front door** — API Gateway is that door
- Supports REST APIs, HTTP APIs, and WebSocket APIs
- Integrates natively with Lambda — zero server management

Architecture — serverless REST API

```
Angular / React / Mobile app
```

```
        │
        │  HTTPS  (SSL free, automatic)
        ▼
API Gateway
  ├─ GET  /users        ──▶  GetUsersLambda    ──▶  DynamoDB
  ├─ POST /users        ──▶  CreateUserLambda  ──▶  DynamoDB
  ├─ GET  /users/{id}   ──▶  GetUserLambda     ──▶  DynamoDB
  └─ POST /orders       ──▶  OrderLambda       ──▶  RDS + SNS


Auth: API key / JWT / Cognito
Rate limiting: 1000 req/sec per stage
CORS: configured once, applies to all routes
```

Create REST API — 5 CLI steps

```
# Step 1: Create the API
aws apigateway create-rest-api \
  --name "UserAPI" \
  --description "User management REST API"
# Returns: { "id": "abc123def" }


# Step 2: Get the root resource ID
aws apigateway get-resources --rest-api-id abc123def
```

```
# Returns root resource id, e.g., "xyz789"



# Step 3: Create /users resource

aws apigateway create-resource \

  --rest-api-id abc123def \

  --parent-id xyz789 \

  --path-part users



# Step 4: Add GET method (no auth for this example)

aws apigateway put-method \

  --rest-api-id abc123def \

  --resource-id <users-resource-id> \

  --http-method GET \

  --authorization-type NONE



# Step 5: Connect to Lambda (AWS_PROXY = Lambda receives full request)

aws apigateway put-integration \

  --rest-api-id abc123def \

  --resource-id <users-resource-id> \

  --http-method GET \

  --type AWS_PROXY \

  --integration-http-method POST \
```

```
  --uri
"arn:aws:apigateway:us-east-1:lambda:path/2015-03-31/functions/arn:aws:lambda:us-east-1:123456789012:function:GetUsers/invocations"



# Deploy to a stage

aws apigateway create-deployment \

  --rest-api-id abc123def \

  --stage-name prod



# Your API is now live at:

# https://abc123def.execute-api.us-east-1.amazonaws.com/prod/users
```

API Gateway with Spring Boot on EC2

```
# application.yml — if running Spring Boot on EC2 behind API Gateway

server:

  port: 8080



# API Gateway forwards requests to:

# http://your-ec2-ip:8080/api/*

# No code changes needed — just configure the integration target
```

> **Free tier:** 1 million API calls/month free for the first 12 months

---

8. CloudWatch — Monitoring & Alerts

What is it?

- **Monitoring, logging, and alerting** for every AWS service and your own app
- **Metrics** — numbers tracked over time (CPU %, error count, request rate, latency)
- **Logs** — application output (your `log.info()` and `log.error()` statements)
- **Alarms** — notify you when a metric crosses a threshold
- **Dashboards** — visual charts of all your metrics in one place

Architecture — alarm pipeline

```
Your app (EC2 / Lambda)
        │
        │  emits metrics automatically (CPU, errors, invocations)
        │  your code adds custom metrics (orders/min)
        ▼
CloudWatch Metrics Store
        │
        │  evaluates: "errors > 10 in last 5 minutes?"
        ▼
Alarm: INSUFFICIENT → OK → ALARM
        │
        │  when state = ALARM
        ▼
SNS Topic  ──▶  Email to team
           ──▶  SMS to on-call engineer
```

```
                ──▶  Trigger Lambda auto-remediation
```

Java — custom metrics and alarms

```
@Service

public class CloudWatchService {



    private final CloudWatchClient cw = CloudWatchClient.builder()

            .region(Region.US_EAST_1)

            .build();



    // ── SEND CUSTOM METRIC ──────────────────────────────────────────

    // Call this every time an order is placed

    public void recordOrderPlaced(String productCategory) {

        cw.putMetricData(PutMetricDataRequest.builder()

            .namespace("MyApp/Orders")              // your custom namespace

            .metricData(

                MetricDatum.builder()

                    .metricName("OrdersPlaced")

                    .value(1.0)

                    .unit(StandardUnit.COUNT)

                    .timestamp(Instant.now())

                    // Dimension: break down by product category
```

```
                    .dimensions(Dimension.builder()
                        .name("Category")
                        .value(productCategory)
                        .build())
                    .build()
            )
            .build());
    }

    // ── SEND LATENCY METRIC ──────────────────────────────────────────
    public void recordApiLatency(String endpoint, long milliseconds) {
        cw.putMetricData(PutMetricDataRequest.builder()
            .namespace("MyApp/API")
            .metricData(
                MetricDatum.builder()
                    .metricName("ResponseTime")
                    .value((double) milliseconds)
                    .unit(StandardUnit.MILLISECONDS)
                    .timestamp(Instant.now())
                    .dimensions(Dimension.builder()
                        .name("Endpoint").value(endpoint).build())
                    .build()
```

```
            )
            .build());
    }

    // ── CREATE ALARM ──────────────────────────────────────────────
    // Alert when error count > 10 in a 5-minute window
    public void createHighErrorAlarm(String snsTopicArn) {
        cw.putMetricAlarm(PutMetricAlarmRequest.builder()
            .alarmName("HighErrorRate")
            .alarmDescription("Fires when errors exceed 10 in 5 minutes")
            .metricName("Errors")
            .namespace("MyApp/Orders")
            .statistic(Statistic.SUM)
            .period(300)                              // 5-minute evaluation window
            .evaluationPeriods(1)                     // 1 window must breach
            .threshold(10.0)
            .comparisonOperator(
                ComparisonOperator.GREATER_THAN_THRESHOLD)
            .alarmActions(snsTopicArn)                // notify this SNS topic
            .okActions(snsTopicArn)                   // also notify when resolved
            .treatMissingData(TreatMissingData.NOT_BREACHING)
```

```
            .build());

    }

}
```

Spring Boot structured logging (automatically ships to CloudWatch on EC2)

```
@Slf4j

@Service

public class OrderService {



    public Order processOrder(OrderRequest request) {

        // Structured log — searchable in CloudWatch Logs Insights

        log.info("Order processing started orderId={} userId={} amount={}",

            request.getId(), request.getUserId(), request.getAmount());



        try {

            Order order = createOrder(request);

            log.info("Order complete orderId={} status=SUCCESS",
order.getId());

            return order;



        } catch (Exception e) {

            log.error("Order failed orderId={} error={} stackTrace={}",

                request.getId(), e.getMessage(), e.getClass().getSimpleName());
```

```
            throw e;

        }

    }

}
```

> **Day-one checklist:** (1) Alarm on error count, (2) log group per service, (3) dashboard with request rate + error rate + latency.
>
> **Free tier:** 10 custom metrics · 10 alarms · 5 GB logs/month — always free

---

9. IAM — Identity & Access Management

What is it?

- **Controls who can do what** in your AWS account
- Never give your app **full admin access** — minimum permissions only
- **Principle of least privilege** — only the actions required, only on specific resources
- **IAM Roles** attach to Lambda/EC2 — no hardcoded keys needed
- **IAM Policies** are JSON documents defining allowed/denied actions

Hierarchy

```
AWS Root Account  (god mode — never use for apps)

        │

        ▼

IAM Role: LambdaRole

        │ has attached →

        ▼

IAM Policy: MyAppPolicy
```

```
        │ allows only →
        ├─ s3:GetObject    on arn:aws:s3:::my-bucket/*
        ├─ s3:PutObject    on arn:aws:s3:::my-bucket/*
        └─ dynamodb:GetItem  on arn:aws:dynamodb:...:table/Users
           dynamodb:PutItem  on arn:aws:dynamodb:...:table/Users
           (NOT dynamodb:DeleteTable — not needed, so not allowed)
```

Create policy + role — 3 CLI steps

```
# Step 1: Create the policy (least-privilege — only what the app needs)
aws iam create-policy \
  --policy-name MyAppPolicy \
  --policy-document '{
    "Version": "2012-10-17",
    "Statement": [
      {
        "Effect": "Allow",
        "Action": ["s3:GetObject", "s3:PutObject"],
        "Resource": "arn:aws:s3:::my-app-bucket/*"
      },
      {
        "Effect": "Allow",
        "Action": ["dynamodb:GetItem", "dynamodb:PutItem",
"dynamodb:UpdateItem"],
```

```
        "Resource": "arn:aws:dynamodb:us-east-1:123456789012:table/Users"

      },

      {

        "Effect": "Allow",

        "Action":
["logs:CreateLogGroup","logs:CreateLogStream","logs:PutLogEvents"],

        "Resource": "*"

      }

    ]

  }'



# Step 2: Create role — allow Lambda to assume it

aws iam create-role \

  --role-name MyLambdaRole \

  --assume-role-policy-document '{

    "Version": "2012-10-17",

    "Statement": [{

      "Effect": "Allow",

      "Principal": { "Service": "lambda.amazonaws.com" },

      "Action": "sts:AssumeRole"

    }]

  }'
```

```
# Step 3: Attach the policy to the role
aws iam attach-role-policy \
  --role-name MyLambdaRole \
  --policy-arn arn:aws:iam::123456789012:policy/MyAppPolicy


# Now assign this role when creating your Lambda:
aws lambda create-function \
  --function-name MyFunction \
  --role arn:aws:iam::123456789012:role/MyLambdaRole \
  ...
```

For EC2 — assign role as instance profile

```
# Create instance profile
aws iam create-instance-profile --instance-profile-name MyEC2Profile


# Add the role to the profile
aws iam add-role-to-instance-profile \
  --instance-profile-name MyEC2Profile \
  --role-name MyEC2Role


# Attach to running instance
```

```
aws ec2 associate-iam-instance-profile \
  --instance-id i-1234567890abcdef0 \
  --iam-instance-profile Name=MyEC2Profile
```

> ⚠️ **Never hardcode AWS access keys in code.** Keys on GitHub are found by bots within seconds. Always use IAM Roles — they provide automatic rotating temporary credentials.
>
> **IAM is always free.**

---

10. CloudFormation — Infrastructure as Code

What is it?

- Define your **entire AWS infrastructure in YAML** — deploy with one command
- Creates EC2, S3, DynamoDB, Lambda, IAM — all at once, **repeatably**
- Same template → dev, staging, prod environments are **guaranteed identical**
- If production breaks: delete the stack and redeploy from the same file in minutes
- Never manage AWS by clicking through the console manually

Template structure

```
# stack.yaml — your entire infrastructure in one file



AWSTemplateFormatVersion: '2010-09-09'
Description: 'My App Stack'



# ── Parameters — customise per environment ──────────────────────
Parameters:
  Environment:
```

```
    Type: String

    Default: dev

    AllowedValues: [dev, staging, prod]



# ── Resources — what to create ──────────────────────────────────

Resources:



  # DynamoDB table

  UsersTable:

    Type: AWS::DynamoDB::Table

    Properties:

      TableName: !Sub 'Users-${Environment}'   # dev → Users-dev, prod → Users-
prod

      BillingMode: PAY_PER_REQUEST

      AttributeDefinitions:

        - AttributeName: userId

          AttributeType: S

      KeySchema:

        - AttributeName: userId

          KeyType: HASH



  # S3 bucket
```

```
  AppBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: !Sub 'my-app-${Environment}-${AWS::AccountId}'


  # IAM Role for Lambda
  LambdaExecutionRole:
    Type: AWS::IAM::Role
    Properties:
      RoleName: !Sub 'LambdaRole-${Environment}'
      AssumeRolePolicyDocument:
        Version: '2012-10-17'
        Statement:
          - Effect: Allow
            Principal: { Service: lambda.amazonaws.com }
            Action: 'sts:AssumeRole'
      Policies:
        - PolicyName: LambdaPolicy
          PolicyDocument:
            Version: '2012-10-17'
            Statement:
              - Effect: Allow
```

```
                Action: [dynamodb:GetItem, dynamodb:PutItem,
dynamodb:UpdateItem]

                Resource: !GetAtt UsersTable.Arn   # ref to table above

              - Effect: Allow

                Action: [logs:CreateLogGroup, logs:CreateLogStream,
logs:PutLogEvents]

                Resource: '*'



  # Lambda function

  UserFunction:

    Type: AWS::Lambda::Function

    Properties:

      FunctionName: !Sub 'UserHandler-${Environment}'

      Runtime: java17

      Handler: com.example.UserHandler::handleRequest

      Role: !GetAtt LambdaExecutionRole.Arn

      MemorySize: 512

      Timeout: 30

      Code:

        S3Bucket: my-deployment-bucket

        S3Key: !Sub 'myapp-${Environment}.jar'

      Environment:

        Variables:
```

```
          TABLE_NAME: !Ref UsersTable           # passes table name as env var
          ENV: !Ref Environment


# ── Outputs — useful values to reference ──────────────────────────
Outputs:
  TableName:
    Value: !Ref UsersTable
    Export:
      Name: !Sub '${Environment}-UsersTableName'
  FunctionArn:
    Value: !GetAtt UserFunction.Arn
```

Deploy commands

```
# Validate the template before deploying
aws cloudformation validate-template --template-body file://stack.yaml


# Deploy for dev
aws cloudformation deploy \
  --template-file stack.yaml \
  --stack-name my-app-dev \
  --parameter-overrides Environment=dev \
  --capabilities CAPABILITY_NAMED_IAM   # required when creating IAM resources
```

```
# Deploy for prod (same template — different parameter)
aws cloudformation deploy \
  --template-file stack.yaml \
  --stack-name my-app-prod \
  --parameter-overrides Environment=prod \
  --capabilities CAPABILITY_NAMED_IAM


# See what WILL change before applying (change set)
aws cloudformation create-change-set \
  --stack-name my-app-prod \
  --template-body file://stack.yaml \
  --change-set-name my-update \
  --capabilities CAPABILITY_NAMED_IAM


# List running stacks
aws cloudformation list-stacks --stack-status-filter CREATE_COMPLETE
UPDATE_COMPLETE


# Delete everything (removes ALL resources in the stack)
aws cloudformation delete-stack --stack-name my-app-dev
```

> **Why use CloudFormation:** One command creates your entire environment. Dev, staging, and

> prod are guaranteed identical. Delete and recreate in minutes. Never drift from your intended state.
>
> **CloudFormation is always free.** You pay only for the resources it creates.

---

11. RDS — Relational Database Service

What is it?

- **Managed MySQL, PostgreSQL, Oracle, or SQL Server** in AWS
- AWS handles: **backups, patching, failover, scaling, monitoring**
- Your Spring Boot + JPA works **completely unchanged** — just update the JDBC URL
- Lives in a **private subnet** — only accessible from inside your VPC, not the internet
- Automatic backups retained for up to **35 days** — restore to any point in time

Architecture

```
Internet ─── BLOCKED (RDS is in private subnet)



EC2 (public subnet)  ──▶  RDS MySQL (private subnet)

                          │

                          ▼

                      Auto-backups to S3

                      Multi-AZ standby (paid feature)

                      Read replicas for reporting
```

Create RDS instance

```
aws rds create-db-instance \

  --db-instance-identifier myapp-db \
```

```
  --db-instance-class db.t3.micro \
  --engine mysql \
  --engine-version 8.0 \
  --master-username admin \
  --master-user-password "MySecurePass123!" \
  --allocated-storage 20 \
  --backup-retention-period 7 \
  --storage-type gp2 \
  --vpc-security-group-ids sg-0123456789abcdef0 \
  --no-publicly-accessible   # ← private — only reachable from VPC


# Check status (takes 5-10 minutes to create)
aws rds describe-db-instances \
  --db-instance-identifier myapp-db \
  --query 'DBInstances[0].{Status:DBInstanceStatus,Endpoint:Endpoint.Address}'
```

Connect Spring Boot — just update application.yml

```
spring:
  datasource:
    # Replace with your RDS endpoint from AWS console
    url: jdbc:mysql://myapp-db.abc123xyz.us-east-1.rds.amazonaws.com:3306/myappdb
    username: admin
```

```
    password: ${DB_PASSWORD}   # use environment variable — never hardcode!
    driver-class-name: com.mysql.cj.jdbc.Driver
    hikari:
      maximum-pool-size: 5     # RDS free tier has connection limits
      connection-timeout: 30000


  jpa:
    hibernate:
      ddl-auto: update         # auto-creates/updates tables from your entities
    show-sql: false
    properties:
      hibernate:
        dialect: org.hibernate.dialect.MySQL8Dialect
```

Your JPA entities work completely unchanged

```
// Same code as local MySQL — nothing changes
@Entity
@Table(name = "users")
public class User {


    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```
    private Long id;



    @Column(nullable = false)

    private String name;



    @Column(unique = true, nullable = false)

    private String email;



    @CreationTimestamp

    private LocalDateTime createdAt;

}



@Repository

public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByEmail(String email);



    @Query("SELECT u FROM User u WHERE u.createdAt >= :since")

    List<User> findRecentUsers(@Param("since") LocalDateTime since);

}
```

RDS vs DynamoDB — which to use

Scenario	Use
Complex joins across multiple tables	**RDS**
ACID transactions	**RDS**
Simple key-value lookups	**DynamoDB**
Massive scale (millions req/sec)	**DynamoDB**
Variable schema items	**DynamoDB**
Existing SQL codebase	**RDS**

> **Free tier:** db.t3.micro (MySQL or PostgreSQL) — 750 hours/month + 20 GB storage for 12 months

---

12. ELB + Auto Scaling

What is it?

- **Elastic Load Balancer (ELB)** — distributes traffic across multiple EC2 instances
- **Auto Scaling Group (ASG)** — automatically adds EC2 when CPU spikes, removes when quiet
- Together: handle **any traffic volume with zero downtime**
- **ALB** (Application Load Balancer) does Layer 7 routing — path-based, host-based
- Health checks ensure traffic goes only to **healthy instances**

Architecture

```
10,000 users/second

        │

        ▼  HTTPS

Application Load Balancer (ALB)
```

```
  ├─ health check: GET /api/health every 30s

  ├─ distributes evenly across all healthy instances

  └─ removes unhealthy instances automatically

        │

   ┌────┼────┐

   ▼    ▼    ▼

EC2-1 EC2-2 EC2-3  ← Auto Scaling Group

(app) (app) (auto-added when CPU > 70%)

        │

        ▼ (all share the same)

    RDS Database
```

Auto Scaling lifecycle — what happens at 70% CPU

```
CPU > 70% detected by CloudWatch

        │

        ▼

CloudWatch Alarm fires → triggers ASG scale-out policy

        │

        ▼

New EC2 launched from Launch Template (same AMI, config)

        │

        ▼  (takes 2-3 minutes)
```

```
EC2 starts, Spring Boot boots, /api/health returns 200
        │
        ▼
ALB health check passes → EC2 added to rotation
        │
        ▼
Traffic now distributed across 3 (or more) instances


CPU drops below 40% → ASG scale-in → terminates extra EC2
```

Set up via CLI — 5 steps

```
# Step 1: Create Application Load Balancer
aws elbv2 create-load-balancer \
  --name myapp-alb \
  --subnets subnet-aaa111 subnet-bbb222 \
  --security-groups sg-0123456789 \
  --type application \
  --scheme internet-facing


# Step 2: Create target group (where ALB sends traffic)
aws elbv2 create-target-group \
  --name myapp-targets \
```

```
  --protocol HTTP \

  --port 8080 \

  --vpc-id vpc-0123456789 \

  --health-check-path /api/health \         # Spring Boot Actuator endpoint

  --health-check-interval-seconds 30 \

  --healthy-threshold-count 2



# Step 3: Create listener (ALB listens on port 443, forwards to target group)

aws elbv2 create-listener \

  --load-balancer-arn <alb-arn> \

  --protocol HTTPS \

  --port 443 \

  --certificates CertificateArn=<acm-cert-arn> \

  --default-actions Type=forward,TargetGroupArn=<target-group-arn>



# Step 4: Create Launch Template (defines each new EC2 that ASG creates)

aws ec2 create-launch-template \

  --launch-template-name myapp-template \

  --launch-template-data '{

    "ImageId": "ami-0c55b159cbfafe1f0",

    "InstanceType": "t3.small",

    "KeyName": "my-key-pair",
```

```
    "SecurityGroupIds": ["sg-0123456789"],

    "IamInstanceProfile": { "Name": "MyEC2Profile" },

    "UserData": "<base64-encoded startup script>"

  }'



# Step 5: Create Auto Scaling Group

aws autoscaling create-auto-scaling-group \

  --auto-scaling-group-name myapp-asg \

  --launch-template LaunchTemplateName=myapp-template,Version='$Latest' \

  --min-size 2 \                          # always at least 2 (high
availability)

  --max-size 10 \                         # never more than 10 instances

  --desired-capacity 2 \                  # start with 2

  --vpc-zone-identifier "subnet-aaa111,subnet-bbb222" \

  --target-group-arns <target-group-arn>



# Add scaling policy: scale out when CPU > 70%

aws autoscaling put-scaling-policy \

  --auto-scaling-group-name myapp-asg \

  --policy-name cpu-scale-out \

  --policy-type TargetTrackingScaling \

  --target-tracking-configuration '{
```

```
    "PredefinedMetricSpecification": {
      "PredefinedMetricType": "ASGAverageCPUUtilization"
    },
    "TargetValue": 70.0,
    "ScaleInCooldown": 300,
    "ScaleOutCooldown": 60
  }'
```

Spring Boot health endpoint (required for ALB health checks)

```
# application.yml — expose health endpoint
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics
  endpoint:
    health:
      show-details: when-authorized
// ALB calls GET /actuator/health every 30 seconds
// Returns {"status":"UP"} when healthy → ALB routes traffic
// Returns {"status":"DOWN"} or 5xx → ALB removes from rotation
// Add spring-boot-starter-actuator to pom.xml — that's it
```

> **Set min-size to 2:** Ensures high availability across two Availability Zones. If one data centre has an outage, the other instance keeps serving traffic.
>
> **Free tier:** 750 hours ELB/month for the first 12 months · Auto Scaling is always free (pay only for EC2 instances)

Quick Reference — All 12 Services

Service	One-line description	When to use	Free tier
EC2	Virtual server	Long-running apps, full OS control	750h/mo 12mo
S3	File storage	Any file: images, PDFs, backups, static websites	5 GB forever
Lambda	Serverless function	Short event-driven tasks, APIs, triggers	1M req forever
DynamoDB	NoSQL database	Key lookups at scale, variable schema, IoT	25 GB forever
SQS	Message queue	Decouple services, reliable async processing	1M req forever
SNS	Pub/Sub broadcast	One event → many consumers simultaneously	1M pub forever
API Gateway	HTTP front door	Public REST API endpoints, Lambda + HTTP routing	1M calls 12mo
CloudWatch	Monitoring + alerts	Metrics, logs, alarms, dashboards	10 metrics forever
IAM	Permissions	Who can do what — roles, policies, users	Always free
CloudFormation	Infrastructure as code	Repeatable environment setup, team consistency	Always free
RDS	Managed SQL	Relational data, joins, complex queries	750h/mo 12mo

Service	One-line description	When to use	Free tier
	database		
ELB + ASG	Load balance + auto-scale	Production-grade availability and scaling	750h ELB 12mo

---

Common Patterns

Pattern 1 — Serverless REST API

```
Client → API Gateway → Lambda → DynamoDB
```

Zero servers. Pay per request. Auto-scales.

Pattern 2 — Async order processing

```
REST API → SQS → Lambda consumer → RDS + Email via SNS
```

Non-blocking. Retryable. Decoupled.

Pattern 3 — High-availability web app

```
Users → ELB → EC2 Auto Scaling Group → RDS (Multi-AZ)

              ↕ static files

              S3 CloudFront CDN
```

Always on. Scales automatically. Zero-downtime deployments.

Pattern 4 — Event fan-out

```
Any service → SNS topic → SQS queue A → email service

                        → SQS queue B → analytics Lambda

                        → SQS queue C → audit logger
```

One event, many independent consumers, each with its own retry.

AWS SDK for Java v2 · Spring Boot 3.x · AWS CLI v2 · Free tier limits verified May 2025

AWS Beginner to Practical Guide · All 12 Core Services · Java SDK v2 · AWS CLI v2 · Spring Boot · Free-tier eligible · May 2025